

Upsell & Revenue Optimization Engine

BRD

1. Summary

A revenue-driving module designed to maximize the lifetime value of each guest . While complex pricing algorithms and specific upsell negotiations are delegated to the PMS and on-site staff respectively, this custom web module acts as the frictionless transactional layer, allowing guests to instantly extend their stays using real-time inventory and pricing data.

2. Scope Definition (Tech vs. PMS vs. Ops)

To ensure lean engineering and avoid duplicating existing systems, the technical scope is strictly defined:

- **IN SCOPE (Custom PWA Tech):**
 - Automated Stay Extension Engine (Inventory check, payment, and auto-confirmation) .
- **OUT OF SCOPE (Handled by eZee PMS Native Logic):**
 - Dynamic Late Checkout Pricing (Next day > 80% occupancy surge, weekend premium pricing, same-day vs. pre-booked pricing differentials) . The PWA will simply fetch the final calculated price from the PMS API.
- **OUT OF SCOPE (Handled by On-Site Ops Team):**
 - Dorm Upsell Logic (Lower bunk upgrade at ₹149, Private room dynamic discounts) .
 - Scarcity Triggers & Notifications ("Limited slots remaining", "Upgrade available") . These will be executed verbally or via manual messaging by the front desk.

3. Use Case: Automated Stay Extension

3.1 Overview

A guest currently staying at Vibe House decides they want to extend their trip by 1 or more days. Instead of going to the front desk, they open the PWA, view the live price (which includes any PMS-generated weekend/surge premiums), and pay instantly to confirm .

3.2 Detailed Action Sequence

Step	User Action (Click/Type)	System Response / Backend Logic

1. Auth & Intent	Guest logs into PWA and clicks "Extend Stay". +1	Validate active in-house session. Retrieve current booking details (Category, Room, Bed).
2. Date Selection	Guest selects the new checkout date.	Call eZee PMS API: Retrieve Room Information for the requested dates.
3. Availability Check	<i>(Wait for processing)</i>	System checks if <i>any</i> bed/room in the guest's current category is available. If none, show "Sold Out" message.
4. "Same Bed" Logic	<i>(Wait for processing)</i>	System checks if the guest's <i>exact current bed</i> is available for the extension dates.
5. Price & Review	View the total cost for the extension.	Fetch dynamic rate from eZee API. UI Disclaimer Trigger: If the <i>exact bed</i> is unavailable (but the category is), display disclaimer: <i>"Extension available! Please note: You may need to move to a different bed/room for the extended dates. Please check with reception for exact location details."</i>

6. Checkout	Click "Proceed to Pay".	Redirect to Razorpay Payment Gateway. Place temporary 10-minute hold on the PMS inventory.
7. Payment	Complete transaction.	Await webhook confirmation.
8. PMS Sync	(Background Process)	Push API request to eZee to update the checkout date on the guest's folio and log the payment.
9. Finish	View "Extension Confirmed" screen.	Generate Staff Task Dashboard notification: <i>"Room 101 Extended. Verify Bed Assignment."</i>

4. Technical Specifications & Business Logic

4.1 "Same Bed" Conflict Resolution (Option B)

The system prioritizes revenue over immediate logistical perfection. If a guest attempts to extend their stay, the system only checks if the overarching *Category* (e.g., 6-Bed Mixed Dorm) has availability.

If another incoming guest has already been assigned to this extending guest's specific bed, the system will **allow the extension to proceed** and collect the payment. The UI disclaimer strictly manages the guest's expectations regarding a potential mid-stay bed transfer.

4.2 Rate Fetching Logic

The custom backend will not calculate surge pricing independently. When the guest requests an extension, the backend will query the eZee PMS for the "Live Walk-in Rate" for that specific date. This ensures that if the eZee Mint system has surged the price due to >80% occupancy or a weekend, the guest pays the optimized premium rate .

4.3 Payment Workflow Requirements

Consistent with the entire PWA architecture, extensions are treated as **strict prepaid transactions**:

- No "Pay at Checkout" option is provided for app-based extensions.
- The PMS folio's departure date is only mutated via API *after* the Razorpay success webhook is received.